

Node.java

.....

```
package com.jlcindia.binarytrees;
```

```
public class Node {
```

```
    int key;
```

```
    Node left;
```

```
    Node right;
```

```
    public Node(int key) {
```

```
        this.key = key;
```

```
        this.left = null;
```

```
        this.right = null;
```

```
    }
```

```
}
```

Lab1.java

.....

```
package com.jlcindia.binarytrees;
```

```
// In Order Traversal
```

```
// Left => Root => Right
```

```
public class Lab1 {
```

```
public static void inorder(Node myroot) {  
    if (myroot != null) {  
        inorder(myroot.left);  
        System.out.println(myroot.key + "\t");  
        inorder(myroot.right);  
    }  
}
```

```
public static void main(String[] args) {  
  
    Node myroot = new Node(10);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(40);  
    myroot.left.right = new Node(50);  
  
    myroot.right.left = new Node(60);  
    myroot.right.right = new Node(70);  
  
    inorder(myroot);  
}
```

Lab2.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Pre Order Traversal
```

```
// Root => Left => Right
```

```
public class Lab2 {
```

```
    public static void preorder(Node myroot) {
```

```
        if (myroot != null) {
```

```
            System.out.println(myroot.key + "\t");
```

```
            preorder(myroot.left);
```

```
            preorder(myroot.right);
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Node myroot = new Node(10);
```

```
        myroot.left = new Node(20);
```

```
        myroot.right = new Node(30);
```

```

        myroot.left.left = new Node(40);
        myroot.left.right = new Node(50);

        myroot.right.left = new Node(60);
        myroot.right.right = new Node(70);

        preorder(myroot);
    }
}

```

Lab3.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Post Order Traversal
```

```
// Left => Right => Root
```

```
public class Lab3 {
```

```
    public static void postorder(Node myroot) {
```

```
        if (myroot != null) {
```

```
            postorder(myroot.left);
```

```
            postorder(myroot.right);
```

```
            System.out.println(myroot.key + "\t");
```

```

        }
    }

    public static void main(String[] args) {

        Node myroot = new Node(10);

        myroot.left = new Node(20);
        myroot.right = new Node(30);

        myroot.left.left = new Node(40);
        myroot.left.right = new Node(50);

        myroot.right.left = new Node(60);
        myroot.right.right = new Node(70);

        postorder(myroot);
    }
}

```

Lab4.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Height of Binary Tree
```

```
public class Lab4 {
```

```
    public static int height(Node myroot) {
```

```
        if (myroot == null) {
```

```
            return 0;
```

```
        }else{
```

```
            int leftHight = height(myroot.left);
```

```
            int rightHight = height(myroot.right);
```

```
            int max= Math.max(leftHight, rightHight);
```

```
            return 1+ max;
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Node myroot = new Node(10);
```

```
        myroot.left = new Node(20);
```

```
        myroot.right = new Node(30);
```

```
        myroot.left.left = new Node(40);
```

```
        myroot.left.right = new Node(50);
```

```
        myroot.right.left = new Node(60);
```

```
        myroot.right.right = new Node(70);
```

```

        myroot.right.right.left= new Node(80);
        myroot.right.right.left.right= new Node(90);

        int height = height(myroot);
        System.out.println(height);

    }
}

```

Lab5.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Size of Binary Tree
```

```
public class Lab5 {
```

```

    public static int getSize(Node myroot) {
        if (myroot == null) {
            return 0;
        } else {
            int leftSize = getSize(myroot.left);
            int rightSize = getSize(myroot.right);

```

```
        return 1 + leftSize + rightSize;
    }
}
```

```
public static void main(String[] args) {
```

```
    Node myroot = new Node(10);
```

```
    myroot.left = new Node(20);
```

```
    myroot.right = new Node(30);
```

```
    myroot.left.left = new Node(40);
```

```
    myroot.left.right = new Node(50);
```

```
    myroot.right.left = new Node(60);
```

```
    myroot.right.right = new Node(70);
```

```
    myroot.right.right.left= new Node(80);
```

```
    myroot.right.right.left.right= new Node(90);
```

```
    int height = getSize(myroot);
```

```
    System.out.println(height);
```

```
}
```

```
}
```

Lab6.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Maximum in Binary Tree
```

```
public class Lab6 {
```

```
    public static int getMax(Node myroot) {  
        if (myroot == null) {  
            return Integer.MIN_VALUE;  
        } else {  
            int leftMax = getMax(myroot.left);  
            int rightMax = getMax(myroot.right);  
            int max = Math.max(leftMax, rightMax);  
            return Math.max(myroot.key, max);  
        }  
    }  
}
```

```
    public static void main(String[] args) {
```

```
        Node myroot = new Node(10);
```

```
        myroot.left = new Node(20);
```

```
myroot.right = new Node(30);
```

```
myroot.left.left = new Node(40);
```

```
myroot.left.right = new Node(50);
```

```
myroot.right.left = new Node(60);
```

```
myroot.right.right = new Node(70);
```

```
// myroot.right.right.left= new Node(80);
```

```
// myroot.right.right.left.right= new Node(90);
```

```
int max = getMax(myroot);
```

```
System.out.println(max);
```

```
}
```

```
}
```

Lab7.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Minimum in Binary Tree
```

```
public class Lab7 {

    public static int getMin(Node myroot) {
        if (myroot == null) {
            return Integer.MAX_VALUE;
        } else {
            int leftMin = getMin(myroot.left);
            int rightMin = getMin(myroot.right);
            int min = Math.min(leftMin, rightMin);
            return Math.min(myroot.key, min);
        }
    }

    public static void main(String[] args) {

        Node myroot = new Node(10);

        myroot.left = new Node(20);
        myroot.right = new Node(30);

        myroot.left.left = new Node(40);
        myroot.left.right = new Node(50);

        myroot.right.left = new Node(60);
        myroot.right.right = new Node(70);
    }
}
```

```

        int min = getMin(myroot);
        System.out.println(min);

    }
}

```

Lab8.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Print the Nodes at K-Distance
```

```
public class Lab8 {
```

```
    public static void printKDistanceNodes(Node myroot,int k) {
```

```
        if(myroot==null) {
```

```
            return;
```

```
        }
```

```
        if(k==0) {
```

```
            System.out.print(myroot.key+"\t");
```

```
        }else {
```

```
            printKDistanceNodes(myroot.left,k-1);
```

```
            printKDistanceNodes(myroot.right,k-1);
```

```
    }  
  
}  
  
public static void main(String[] args) {  
  
    Node myroot = new Node(10);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(40);  
    myroot.left.right = new Node(50);  
  
    myroot.right.left = new Node(60);  
    myroot.right.right = new Node(70);  
  
    printKDistanceNodes(myroot,0);  
  
}  
}
```

Lab9.java

.....

```
package com.jlcindia.binarytrees;
```

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
// Level Order Traversal( Breadth First)
```

```
//Left -> Right
```

```
public class Lab9 {
```

```
    public static void levelOrder(Node myroot) {
```

```
        if(myroot==null) {
```

```
            return;
```

```
        }
```

```
        Queue<Node> myque = new LinkedList<Node>();
```

```
        myque.add(myroot);
```

```
        while(!myque.isEmpty()) {
```

```
            Node currNode = myque.poll();
```

```
            System.out.print(currNode.key+"\t");
```

```
            if(currNode.left!=null)
```

```
                myque.add(currNode.left);
```

```

        if(currNode.right!=null)
            myque.add(currNode.right);

    }

}

public static void main(String[] args) {

    Node myroot = new Node(10);

    myroot.left = new Node(20);
    myroot.right = new Node(30);

    myroot.left.left = new Node(40);
    myroot.left.right = new Node(50);

    myroot.right.left = new Node(60);
    myroot.right.right = new Node(70);

    levelOrder(myroot);

}
}

```

Lab10.java

.....

```
package com.jlcindia.binarytrees;
```

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
// Level Order Traversal( Breadth First)
```

```
// Right -> Left
```

```
public class Lab10 {
```

```
    public static void levelOrder(Node myroot) {
```

```
        if (myroot == null)
```

```
            return;
```

```
        Queue<Node> myqueue = new LinkedList<Node>();
```

```
        myqueue.add(myroot);
```

```
        while (!myqueue.isEmpty()) {
```

```
            Node currNode = myqueue.poll();
```

```
            System.out.print(currNode.key + "\t");
```

```
            if (currNode.right != null)
```

```
                myqueue.add(currNode.right);
```

```

        if (currNode.left != null)
            myque.add(currNode.left);
    }

}

public static void main(String[] args) {

    Node myroot = new Node(10);

    myroot.left = new Node(20);
    myroot.right = new Node(30);

    myroot.left.left = new Node(40);
    myroot.left.right = new Node(50);

    myroot.right.left = new Node(60);
    myroot.right.right = new Node(70);

    levelOrder(myroot);

}
}

```

Lab11.java

.....

```
package com.jlcindia.binarytrees;
```

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
// Level Order Traversal ( Breadth First)
```

```
// Level by Level or Line by Line
```

```
public class Lab11 {
```

```
    public static void levelOrder(Node myroot) {
```

```
        if (myroot == null) {
```

```
            return;
```

```
        }
```

```
        Queue<Node> myqueue = new LinkedList<Node>();
```

```
        myqueue.add(myroot);
```

```
        while (!myqueue.isEmpty()) {
```

```
            int count = myqueue.size();
```

```
            for (int i = 1; i <= count; i++) {
```

```
                Node currNode = myqueue.poll();
```

```

        System.out.print(currNode.key + "\t");

        if (currNode.left != null)
            myque.add(currNode.left);

        if (currNode.right != null)
            myque.add(currNode.right);
    } //end of For

    System.out.println("");
} //end while

}

```

```

public static void main(String[] args) {

    Node myroot = new Node(10);

    myroot.left = new Node(20);
    myroot.right = new Node(30);

    myroot.left.left = new Node(40);
    myroot.left.right = new Node(50);

    myroot.right.left = new Node(60);
    myroot.right.right = new Node(70);
}

```

```
        levelOrder(myroot);  
    }  
}
```

Lab12.java

.....

```
package com.jlcindia.binarytrees;
```

```
import java.util.Stack;
```

```
// InOrder -Iterative Style
```

```
public class Lab12 {
```

```
    public static void inOrderIterative(Node myroot) {
```

```
        if (myroot == null) {  
            return;  
        }
```

```
        Stack<Node> mystack = new Stack<Node>();
```

```
        Node currNode = myroot;
```

```
while (currNode != null || !mystack.isEmpty()) {  
  
    while (currNode != null) {  
        mystack.push(currNode);  
        currNode = currNode.left;  
    }  
  
    currNode = mystack.pop();  
    System.out.print(currNode.key+"\t");  
    currNode = currNode.right;  
}  
  
}
```

```
public static void main(String[] args) {  
  
    Node myroot = new Node(10);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(40);  
    myroot.left.right = new Node(50);  
  
    myroot.right.left = new Node(60);  
    myroot.right.right = new Node(70);  
}
```

```

        inOrderIterative(myroot);

    }
}

```

Lab13.java

.....

```
package com.jlcindia.binarytrees;
```

```
import java.util.Stack;
```

```
// PreOrder -Iterative Style
```

```
// Root -> Left -> Right
```

```
public class Lab13 {
```

```
    public static void preOrderIterative(Node myroot) {
```

```
        if (myroot == null)
```

```
            return;
```

```
        Stack<Node> mystack = new Stack<Node>();
```

```
        mystack.push(myroot);
```

```
while (!mystack.isEmpty()) {  
    Node currNode = mystack.pop();  
    System.out.print(currNode.key + "\t");  
  
    if (currNode.right != null)  
        mystack.push(currNode.right);  
  
    if (currNode.left != null)  
        mystack.push(currNode.left);  
  
}  
  
}
```

```
public static void main(String[] args) {  
  
    Node myroot = new Node(10);  
  
    myroot.left = new Node(20);  
    myroot.right = new Node(30);  
  
    myroot.left.left = new Node(40);  
    myroot.left.right = new Node(50);  
  
    myroot.right.left = new Node(60);  
    myroot.right.right = new Node(70);  
}
```

```
        preOrderIterative(myroot);  
    }  
}
```

Lab14.java

.....

```
package com.jlcindia.binarytrees;
```

```
import java.util.Stack;
```

```
// PostOrder -Iterative Style
```

```
// Left -> Right -> Root
```

```
public class Lab14 {
```

```
    public static void postOrderIterative(Node myroot) {
```

```
        if (myroot == null)
```

```
            return;
```

```
        Stack<Integer> output = new Stack<Integer>();
```

```
        Stack<Node> mystack = new Stack<Node>();
```

```
mystack.push(myroot);

while (!mystack.isEmpty()) {
    Node currNode = mystack.pop();
    output.push(currNode.key);

    if (currNode.left != null)
        mystack.push(currNode.left);

    if (currNode.right != null)
        mystack.push(currNode.right);
}
while (!output.isEmpty()) {
    System.out.print(output.pop()+"\t");
}
}
```

```
public static void main(String[] args) {

    Node myroot = new Node(10);

    myroot.left = new Node(20);
    myroot.right = new Node(30);

    myroot.left.left = new Node(40);
    myroot.left.right = new Node(50);
```

```

        myroot.right.left = new Node(60);
        myroot.right.right = new Node(70);

        postOrderIterative(myroot);

    }
}

```

Lab15.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Checking for Full Binary Tree
```

```
public class Lab15 {
```

```
    public static boolean isFullBinaryTree(Node mynode) {
```

```
        // 1. Null Tree
```

```
        if (mynode == null) {
```

```
            return true;
```

```
        }
```

```

// 2.When Single Root is there
if (mynode.left == null && mynode.right == null) {
    return true;
}

// 3.When Childrens are there
if (mynode.left != null && mynode.right != null) {
    boolean leftFlag = isFullBinaryTree(mynode.left);
    boolean rightFlag = isFullBinaryTree(mynode.right);
    return ( leftFlag && rightFlag);
}

return false;
}

public static void main(String[] args) {

    Node myroot = new Node(10);

    myroot.left = new Node(20);
    myroot.right = new Node(30);

    myroot.left.left = new Node(40);
    myroot.left.right = new Node(50);

    myroot.right.left = new Node(60);

```

```

        //myroot.right.right = new Node(70);

        boolean flag = isFullBinaryTree(myroot);
        System.out.println(flag);

    }
}

```

Lab16.java

.....

```
package com.jlcindia.binarytrees;
```

```
// Checking for Balanced Binary Tree
```

```
public class Lab16 {
```

```
    public static int height(Node myroot) {
```

```
        if (myroot == null) {
```

```
            return 0;
```

```
        }else{
```

```
            int leftHight = height(myroot.left);
```

```
            int rightHight = height(myroot.right);
```

```

        int max= Math.max(leftHight, rightHight);
        return 1+ max;
    }
}

```

```

public static boolean isBalanced(Node mynode) {

    // 1. Null Tree
    if (mynode == null) {
        return true;
    }

    int leftHeight = height(mynode.left);
    int rightHeight = height(mynode.right);
    int diff = Math.abs(leftHeight-rightHeight);

    return ( diff <= 1 && isBalanced(mynode.left) &&
isBalanced(mynode.right));
}

```

```

public static void main(String[] args) {

    Node myroot = new Node(10);

    myroot.left = new Node(20);
    myroot.right = new Node(30);
}

```

```
myroot.left.left = new Node(40);
```

```
myroot.left.right = new Node(50);
```

```
myroot.right.left = new Node(60);
```

```
myroot.right.left.left = new Node(70);
```

```
myroot.right.left.left.left = new Node(80);
```

```
boolean flag = isBalanced(myroot);
```

```
System.out.println(flag);
```

```
}
```

```
}
```